

Daryl D. Silva

BSCpE-4B2

Laboratory Activity No. 3:

Topic belongs to: Programming Constructs and Paradigms

Title: Implementing CRUD Operations for Books and Users in the Library Management System

Introduction:

This activity involves implementing CRUD (Create, Read, Update, Delete) operations for books and users in the Library Management System. You will create views and forms to handle these operations via a web interface.

Objectives:

- Create views for CRUD operations.
- Use Django forms to manage user input.
- Learn about Django's URL routing for view handling

Results:

The user can now view the list of books, add new books, update existing books, and delete books.

Follow-Up Questions:

1. How do Django forms work with models for CRUD operations?

Answer: Django forms, especially ModelForms, simplify CRUD operations by mapping form fields directly to model fields. They handle data validation and saving automatically.
Example:

```
from django.forms import.ModelForm  
from .models import Book
```

```
class BookForm(ModelForm):
```

```
    class Meta:
```

```
        model = Book
```

```
        fields = ['title', 'author', 'isbn']
```

- Create & Update: form.save() saves form data to the database.
- Read: Retrieve data with Model.objects.all().
- Delete: Use object.delete().

2. What is the role of the redirect() function in Django views?

Answer: The redirect() function in Django redirects users to a different URL after processing a request, typically after form submission or a successful action. Example:

```
from django.shortcuts import redirect
```

```
def book_added(request):
```

```
    return redirect('book_list') # Redirects to the book list page
```

It improves user experience by preventing duplicate form submissions and ensuring smooth navigation.

Findings:

- CRUD operations (Create, Read, Update, Delete) enable efficient management of books and users in the Library Management System.
- Django's ModelForms simplify form handling by mapping model fields to form fields.
- Views and URL patterns handle user interactions, while Django's ORM executes database operations seamlessly.
- The redirect() function ensures smooth navigation after successful actions, preventing duplicate submissions.

Summary:

Implementing CRUD operations in Django allows users to manage books and user records efficiently. ModelForms streamline form handling, while Django's ORM simplifies database interactions. Proper URL routing and views ensure smooth data manipulation.

Conclusion:

A well-structured CRUD implementation enhances the Library Management System's functionality, making book and user management more efficient. By leveraging Django's built-in tools, developers can build scalable and maintainable applications.